Práctica 2 Grupo B9

Francisco Javier Martín Lunas Escobar

Pedro Jesús Cros Pérez

Index

Index **1**
1. Correcciones P1 ..... **2**
2. Refactorización Arquitectónica **3**
El problema: lógica atrapada en main() 3
La solución: extraer la lógica a core_logic 3
Por qué run_simulation acepta parámetros con valores por defecto 5
3. Auditoría de Test Generados **6**
Caso 1: Happy Path - RunSimulation_HappyPath 7
Caso 2: Edge Case — RunSimulation_EdgeCase_EmptyFiles 8
Caso 3: Error Handling — RunSimulation_ErrorHandling_MissingFiles 9

Francisco Javier Martín Lunas Escobar
Pedro Jesús Cros Pérez

1. Correcciones P1

Hemos realizado los siguientes cambios:

1. Hemos agregado número de página en el documento
2. Hemos agregado más texto explicativo en el documento
3. Hemos agregado el fichero de Doxyfile en el zip

Francisco Javier Martín Lunas Escobar
Pedro Jesús Cros Pérez

2. Refactorización Arquitectónica

El problema: lógica atrapada en main()

En la Práctica 1, el fichero **main.cpp** contenía directamente toda la lógica del programa: las pruebas de **ListaEnlazada<int>**, las consultas a **MediExpress**, las funciones auxiliares **mostrarLista()** y **separador()**, y los propios **#include** de todas las cabeceras. Esto cumplía funcionally, pero presentaba un problema de testabilidad: la función **main()** es el punto de entrada único del ejecutable y no puede ser invocada por ningún framework de pruebas automatizado como Google Test. En consecuencia, toda esa lógica era imposible de verificar de forma automática.

La solución: extraer la lógica a core_logic

La refactorización consistió en separar las responsabilidades en tres ficheros con roles bien definidos:

src/main.cpp — punto de entrada mínimo

Su única responsabilidad ahora es arrancar el programa y devolver el control al sistema operativo. Incluye únicamente **core_logic.h** e invoca **run_simulation()**. Tiene 20 líneas frente a las más de 200 originales

```
src > main.cpp > ...
/**
 * @file main.cpp
 * @author Francisco Javier Martín-Lunas Escobar (fjme0008@red.ujaen.es)
 * @brief Punto de entrada del programa. Delega toda la lógica en run_simulation()
 * @version 0.2
 * @date 2026-03-22
 *
 * @copyright Copyright (c) 2026
 */

#include "../include/core_logic.h"

/**
 * @brief Función principal. Inicia el programa y devuelve el control al sistema
 * @return 0 si la ejecución finaliza correctamente.
 */
int main() {
    run_simulation();
    return 0;
}
```

include/core_logic.h — contrato público

Centraliza todos los **#include** necesarios (**<iostream>**, **<fstream>**, **<string>**, **<sstream>** y las cabeceras del dominio), y declara las dos funciones que exporta el módulo:

- **bool safe_read(unsigned int& value)** — lectura segura de enteros desde std::cin.

- **void run_simulation(...)** — función principal con parámetros por defecto para las rutas de los CSV, lo que permite a los tests inyectar rutas propias sin tocar los datos reales.

```
include > h core_logic.h > ...
1  > /** ...
10
11   #pragma once
12
13   #include <iostream>
14   #include <fstream>
15   #include <string>
16   #include <sstream>
17
18   #include "VDinamico.h"
19   #include "ListaEnlazada.h"
20   #include "PaMedicamento.h"
21   #include "MediExpress.h"
22
23 > /** ...
33   bool safe_read(unsigned int& value);
34
35   /**
36    * @brief Ejecuta la simulación completa del programa.
37    *
38    * Contiene toda la lógica que antes residía en main(): pruebas de ListaEnlazada<int>
39    * y operaciones sobre el sistema MediExpress. Al estar desacoplada de main(), esta
40    * función puede ser invocada y verificada por el framework de pruebas Google Test.
41    *
42    * Los parámetros tienen valores por defecto para que main() no necesite cambios.
43    * Los tests pasan rutas propias para no tocar nunca los datos reales del proyecto.
44    *
45    * @param medsPath Ruta al CSV de medicamentos.
46    * @param labsPath Ruta al CSV de laboratorios.
47    */
48   void run_simulation(
49       const std::string& medsPath = "../data/pa_medicamentos.csv",
50       const std::string& labsPath = "../data/laboratorios.csv"
51   );
```

src/core_logic.cpp — toda la lógica de negocio
Contiene la implementación de las tres funciones:

- **mostrarLista()** y **separador()** se declaran como **static** (visibles solo dentro de este fichero, pues son detalles internos).
- **safe_read()** gestiona errores de formato y EOF para que los tests automatizados no queden bloqueados en un bucle infinito ante entradas inesperadas.
- **run_simulation()** alberga íntegramente el código que antes estaba en **main()**: las pruebas de **ListaEnlazada<int>** (Prueba I) y las operaciones sobre **MediExpress** (Prueba II), envueltas en un bloque **try/catch**.

```
src > core_logic.cpp > run_simulation(const std::string &, const std::string &)
11  #include "../include/core_logic.h"
12 > /** ...
16  static void mostrarLista(const ListaEnlazada<int> &lista) {
17      auto it = lista.iterator();
18      std::cout << "[ ";
19      while (!it.isEnd()) {
20          std::cout << it.data() << " ";
21          it.next();
22      }
23      std::cout << "]" << std::endl;
24  }
25
26 > /** ...
30  static void separador(const std::string &titulo) {
31      std::cout << "==========================================================================================" <<
32      std::cout << "------------------------------------------- " << titulo << " -------------------------------------------" << std::endl;
33      std::cout << "==========================================================================================" << std::endl;
34  }
35
36  bool safe_read(unsigned int& value) {
37      if (std::cin >> value) return true; // Lectura correcta
38
39      // Comprobar si se alcanzó el fin de fichero
40      if (std::cin.eof()) return false; // EOF: terminar el programa
41
42      // Error de formato (p.ej. texto en lugar de número)
43      std::cin.clear(); // Limpiar el estado de error
44      std::cin.ignore(10000, '\n'); // Descartar el resto de la línea
45      return true; // Continuar el programa
46  }
47
48  void run_simulation(const std::string& medsPath, const std::string& labsPath) {
49      try {
50          std::cout << "==========================================================================================" << std::endl;
51          std::cout << "------------------------------------------- " << "Prueba I" << " -------------------------------------------" << std::endl;
52          std::cout << "==========================================================================================" << std::endl;
```

Por qué **run_simulation** acepta parámetros con valores por defecto

La función se declaró como:

```
48  void run_simulation(
49      const std::string& medsPath = "../data/pa_medicamentos.csv",
50      const std::string& labsPath = "../data/laboratorios.csv"
51  );
```

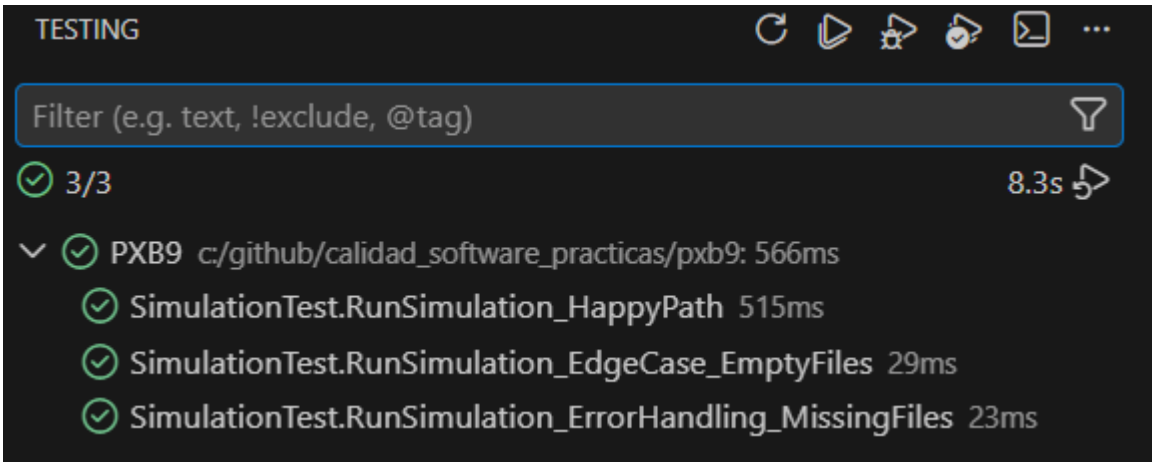
Esto permite que **main()** la llame sin argumentos (comportamiento original inalterado) y que los tests de Google Test la invoquen pasando rutas a ficheros de datos de prueba controlados, garantizando que las pruebas son reproducibles y no dependen del entorno de ejecución.

```
Commit d951066                                                    Browse files
xavivi8 committed 2 days ago
Update main.cpp Use core_logic
main                                                 1 parent 00cafee commit d951066

Filter files...                  1 file changed  +6 -192 lines changed   Search within code
PXB9/src                         PXB9/src/main.cpp                                  +6 -192
  main.cpp
                                 @@ -1,206 +1,20 @@
                                 1  1  /**
                                 2  2   * @file main.cpp
                                 3  3   * @author Francisco Javier Martín-Lunas Escobar (fjme0008@red.ujaen.es)
                                 4     - * @brief Punto de entrada del programa. Pruebas de ListaEnlazada y MediExpress.
                                 5     - * @version 0.1
                                 6     - * @date 2025-10-08
                                    4  + * @brief Punto de entrada del programa. Delega toda la lógica en run_simulation().
                                    5  + * @version 0.2
                                    6  + * @date 2026-03-22
                                 7  7   *
                                 8  8   * @copyright Copyright (c) 2026
                                 9  9   */
                                 10    - #include <iostream>
                                 11    - #include <fstream>
                                 12    - #include <string>
                                 13    - #include <sstream>
                                 14    - #include "../include/VDinamico.h"
                                 15    - #include "../include/ListaEnlazada.h"
```

Francisco Javier Martín Lunas Escobar
Pedro Jesús Cros Pérez

3. Auditoría de Test Generados



Caso 1: Happy Path - RunSimulation_HappyPath

Objetivo: verificar que run_simulation() se ejecuta correctamente de principio a fin cuando recibe datos de entrada válidos y bien formados.

Funcionamiento:

1. *createValidFiles()* crea dos ficheros CSV con formato correcto (separador ;, campos completos): 3 medicamentos y 2 laboratorios (uno en Granada, otro en Madrid).
2. Se llama a *run_simulation(TEST_MEDS, TEST_LABS)*.
3. Se captura la salida y se verifican tres aserciones con *EXPECT_NE(...,std::string::npos)*:
    a. Que la **Prueba I** (ListaEnlazada) imprime *"Todos los pasos se ejecutaron correctamente."* → confirma que todas las operaciones de inserción, búsqueda y borrado en la lista se completaron sin errores
    b. Que la **Prueba II** imprime el separador *"Labs Granada"* → confirma que MediExpress cargó los datos y ejecutó la búsqueda por ciudad
    c. Que aparece *"Labs Madrid, 10 primeros"* → confirma que la búsqueda de Madrid también se ejecutó
4. Se verifica que *std::cerr* está vacío → ningún error inesperado.

En resumen: es la prueba de la "ruta feliz". Si falla, significa que la lógica principal está rota con datos correctos.

```
[main] Building folder: c:/GitHub/calidad_software_practicas/PXB9/build
[build] Starting build
[proc] Executing command: C:\mingw64\bin\cmake.EXE --build c:/GitHub/calidad_software_practicas/PXB9/build --config Debug
--target all -j 16 --
[build] [ 36%] Built target MiPrograma
[build] [ 40%] Built target gtest
[build] [ 50%] Built target gtest_main
[build] [ 59%] Built target gmock
[build] [ 90%] Built target MisTests
[build] [100%] Built target gmock_main
[driver] Build completed: 00:00:01.079
[build] Build finished with exit code 0
[proc] Executing command: C:\mingw64\bin\ctest.exe -j16 -C Debug -T test --output-on-failure -R ^SimulationTest\.
RunSimulation_HappyPath$
[ctest] Cannot find file: C:/GitHub/calidad_software_practicas/PXB9/build/DartConfiguration.tcl
[ctest] Cannot find file: C:/GitHub/calidad_software_practicas/PXB9/build/DartConfiguration.tcl
[ctest] Test project C:/GitHub/calidad_software_practicas/PXB9/build
[ctest]     Start 1: SimulationTest.RunSimulation_HappyPath
[ctest] 1/1 Test #1: SimulationTest.RunSimulation_HappyPath ...   Passed    0.03 sec
[ctest]
[ctest] 100% tests passed, 0 tests failed out of 1
[ctest]
[ctest] Total Test time (real) =   0.08 sec
[ctest] CTest finished with return code 0
```

Caso 2: Edge Case — RunSimulation_EdgeCase_EmptyFiles

Objetivo: verificar que *run_simulation()* se comporta de forma controlada y sin cuelgues cuando los ficheros CSV existen pero están completamente vacíos (cero registros).

Funcionamiento:

1. createEmptyFiles() crea los dos ficheros CSV con contenido vacío ("").
2. Se llama a run_simulation(TEST_MEDS, TEST_LABS).
3. Se verifican cuatro aserciones:
    a. Que la **Prueba I** sigue completándose correctamente (*"Todos los pasos se ejecutaron correctamente."*) → la lista enlazada no depende de los CSVs, por lo que debe funcionar igualmente
    b. Que *"Medicamentos sin laboratorio: 0"* aparece en la salida → MediExpress cargó 0 medicamentos, resultado coherente con entrada vacía.
    c. Que *"Laboratorios en Granada o provincia: 0"* aparece → búsqueda sobre conjunto vacío da 0 resultados.
    d. Que *"Laboratorios en Madrid o provincia: 0"* aparece → ídem para Madrid.
4. *std::cerr* debe seguir vacío → un fichero vacío pero existente se abre sin error; no es un caso de fallo.

En resumen: prueba los límites del sistema. Detectaría crashes por acceso a posiciones nulas, divisiones por cero o bucles sobre colecciones vacías

```
[main] Building folder: c:/GitHub/calidad_software_practicas/PXB9/build
[build] Starting build
[proc] Executing command: C:\mingw64\bin\cmake.EXE --build c:/GitHub/calidad_software_practicas/PXB9/build --config Debug
--target all -j 16 --
[build] [ 31%] Built target MiPrograma
[build] [ 40%] Built target gtest
[build] [ 50%] Built target gtest_main
[build] [ 59%] Built target gmock
[build] [ 68%] Built target gmock_main
[build] [100%] Built target MisTests
[driver] Build completed: 00:00:01.105
[build] Build finished with exit code 0
[proc] Executing command: C:\mingw64\bin\ctest.exe -j16 -C Debug -T test --output-on-failure -R ^SimulationTest\.
RunSimulation_EdgeCase_EmptyFiles$
[ctest] Cannot find file: C:/GitHub/calidad_software_practicas/PXB9/build/DartConfiguration.tcl
[ctest] Cannot find file: C:/GitHub/calidad_software_practicas/PXB9/build/DartConfiguration.tcl
[ctest] Test project C:/GitHub/calidad_software_practicas/PXB9/build
[ctest]     Start 2: SimulationTest.RunSimulation_EdgeCase_EmptyFiles
[ctest] 1/1 Test #2: SimulationTest.RunSimulation_EdgeCase_EmptyFiles ...   Passed    0.02 sec
[ctest]
[ctest] 100% tests passed, 0 tests failed out of 1
[ctest]
[ctest] Total Test time (real) =   0.07 sec
[ctest] CTest finished with return code 0
```

Caso 3: Error Handling — RunSimulation_ErrorHandling_MissingFiles

Objetivo: verificar que run_simulation() detecta y reporta correctamente el error cuando los ficheros CSV no existen, terminando de forma controlada sin crash ni excepción no capturada.

Funcionamiento:

1. No se llama a ningún método de creación de ficheros: *TEST_MEDS* y *TEST_LABS* apuntan a rutas que no existen en disco.
2. Se llama a *run_simulation(TEST_MEDS, TEST_LABS)*.
3. Internamente, *MediExpress* intenta abrir los ficheros y al no encontrarlos, el método *loadLabFromCsv()* emite un mensaje de error por *std::cerr*.
4. La única aserción es *EXPECT_FALSE(cerr_buffer.str().empty())* → se verifica que *std::cerr no está vacío*, es decir, que la función reportó el fallo.
5. El hecho de que el test termine normalmente (sin SEGFAULT ni excepción no capturada) confirma que el manejo de errores es robusto gracias al bloque *try/catch* de *run_simulation()*.

En resumen: prueba la resiliencia del sistema ante entradas inválidas graves. Si fallara, indicaría que el programa se bloquea silenciosamente o lanza una excepción no controlada ante ficheros inexistentes.

```
[main] Building folder: c:/GitHub/calidad_software_practicas/PXB9/build
[build] Starting build
[proc] Executing command: C:\mingw64\bin\cmake.EXE --build c:/GitHub/calidad_software_practicas/PXB9/build --config Debug
--target all -j 16 --
[build] [ 31%] Built target MiPrograma
[build] [ 40%] Built target gtest
[build] [ 50%] Built target gtest_main
[build] [ 59%] Built target gmock
[build] [ 68%] Built target gmock_main
[build] [100%] Built target MisTests
[driver] Build completed: 00:00:01.113
[build] Build finished with exit code 0
[proc] Executing command: C:\mingw64\bin\ctest.exe -j16 -C Debug -T test --output-on-failure -R ^SimulationTest\.
RunSimulation_ErrorHandling_MissingFiles$
[ctest] Cannot find file: C:/GitHub/calidad_software_practicas/PXB9/build/DartConfiguration.tcl
[ctest] Cannot find file: C:/GitHub/calidad_software_practicas/PXB9/build/DartConfiguration.tcl
[ctest] Test project C:/GitHub/calidad_software_practicas/PXB9/build
[ctest]     Start 3: SimulationTest.RunSimulation_ErrorHandling_MissingFiles
[ctest] 1/1 Test #3: SimulationTest.RunSimulation_ErrorHandling_MissingFiles ...   Passed    0.02 sec
[ctest]
[ctest] 100% tests passed, 0 tests failed out of 1
[ctest]
[ctest] Total Test time (real) =   0.03 sec
[ctest] CTest finished with return code 0
```